

VIT BHOPAL UNIVERSITY

VITyarthi – Build Your Own Project

PROJECT REPORT

SMART WAREHOUSE STOCK OPTIMIZATION SYSTEM

Java CLI Version

A Console-Based Inventory Management and
Stock Optimization Application

Name: Atharva Kulkarni

Registration Number: 25BAI10584

Course: CSE2006 – Programming in Java

Academic Year:

2026–2027

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope of the Project
6. Target Users
7. Functional Requirements
8. Non-Functional Requirements
9. System Features and Modules
10. Technology Stack
11. System Architecture
12. System Workflow
13. System Design Diagrams
 - 13.1 Use Case Diagram
 - 13.2 Sequence Diagram
 - 13.3 Class Diagram
 - 13.4 ER Diagram
14. Database Design
15. Object-Oriented Design
16. CSE2006 Concept Mapping
17. Smart Reorder Recommendation Algorithm
18. Multithreading and Synchronization
19. Exception Handling and Input Validation
20. File I/O and Report Generation
21. Implementation Details
22. Testing and Validation
23. Running the Project and Results
24. Design Decisions
25. Challenges Faced
26. Limitations
27. Future Enhancements
28. Learning Outcomes
29. Conclusion
30. References

1. ABSTRACT

The **Smart Warehouse Stock Optimization System** is a Java 25 command-line application designed to manage warehouse inventory, suppliers, stock transactions, reorder recommendations, analytics, and reports.

Traditional inventory management can become inefficient when stock quantities are maintained manually. Errors in stock records can lead to insufficient inventory, unnecessary stock accumulation, invalid dispatches, and delayed replenishment decisions.

The proposed system provides a centralized local solution using **Java, JDBC, and an embedded H2 database**. It supports product and supplier management, stock-in, stock-out, returns, adjustments, transaction history, analytics, automated reorder recommendations, report generation, and background inventory monitoring.

The project demonstrates important concepts from the CSE2006 Programming in Java course, including object-oriented programming, inheritance, polymorphism, interfaces, enumerations, exception handling, collections, streams, Java I/O, JDBC, multithreading, synchronization, and database transactions.

The system is implemented as a **CLI/console version** to provide a lightweight and self-contained Java application suitable for local execution without requiring an external database server or web application.

2. INTRODUCTION

Warehouse inventory management required constantly tracking products, stock quantities, suppliers, stock movements, and replenishment requirements

A warehouse may contain hundreds or thousands of products. If stock levels are not monitored properly, products may become unavailable when required. On the other hand, excessive stock can increase storage costs and reduce operational efficiency.

The Smart Warehouse Stock Optimization System addresses these problems by providing an integrated inventory management application.

The system stores warehouse data in an embedded H2 database and provides a menu-driven command-line interface. Users can perform inventory operations, view transaction history, analyze stock levels, generate reports, and obtain automated reorder recommendations.

The project also demonstrates how Java programming concepts can be applied to a practical warehouse-management problem.

3. PROBLEM STATEMENT

Manual warehouse inventory management can result in:

- Incorrect stock quantities.
- Difficulty tracking stock movement.
- Invalid stock-out operations.
- Incomplete transaction history.
- Delayed identification of low-stock products.
- Difficulty determining when products should be reordered.
- Lack of centralized reporting and analytics.
- Potential inconsistencies when multiple stock operations occur concurrently.

The proposed system provides a local computerized solution that maintains product and supplier information, records stock transactions, validates inventory operations, calculates reorder recommendations, generates reports, and provides warehouse analytics.

4. OBJECTIVES

The major objectives of the project are:

1. To develop a Java-based warehouse inventory management system.
 2. To provide persistent local storage using an embedded H2 database.
 3. To manage products and suppliers efficiently.
 4. To support stock-in, stock-out, return, and adjustment operations.
 5. To maintain a complete stock transaction history.
 6. To validate inventory operations and prevent invalid stock movements.
 7. To calculate intelligent reorder recommendations.
 8. To consider stock levels, safety stock, demand history, and supplier lead time.
 9. To provide warehouse analytics.
 10. To generate CSV and text-based reports.
 11. To demonstrate multithreading and synchronization.
 12. To apply major concepts from the CSE2006 Programming in Java syllabus.
-

5. SCOPE OF THE PROJECT

5.1 Included Scope

The system supports:

- Product management.
- Supplier management.
- Product search and filtering.
- Inventory management.
- Stock-in operations.
- Stock-out operations.
- Product returns.
- Stock adjustments.
- Transaction history.
- Smart reorder recommendations.
- Warehouse analytics.
- CSV report generation.
- Text report generation.
- Background inventory monitoring.
- Input validation.
- Exception handling.
- Local database persistence.

5.2 Excluded Scope

The current CLI version does not include:

- User authentication.
 - Role-based access control.
 - Web APIs.
 - GUI/frontend.
 - External database servers.
 - Cloud deployment.
 - Multiple warehouse locations.
 - Machine-learning-based demand forecasting.
 - External supplier integrations.
-

6. TARGET USERS

The system is designed primarily for:

Warehouse Supervisors

Can monitor inventory levels, stock movements, and warehouse statistics.

Inventory Controllers

Can maintain product information and process inventory transactions.

Procurement Staff

Can use reorder recommendations to identify products requiring replenishment.

7. FUNCTIONAL REQUIREMENTS

FR-1 Product Management

The system shall allow users to:

- Add new products.
- View all products.
- Search products by name.
- Search products by SKU.
- Filter products by category.
- Update product details.
- Delete products.

FR-2 Supplier Management

The system shall allow users to:

- Add suppliers.
- View suppliers.
- Search suppliers.
- Update supplier information.
- Delete suppliers.

FR-3 Inventory Management

The system shall support:

- Stock-in.
- Stock-out.
- Returns.
- Stock adjustments.

The system shall prevent a stock-out quantity greater than the available quantity.

FR-4 Transaction Management

The system shall:

- Record inventory movements.
- Store transaction type.
- Store quantity changes.
- Maintain transaction history.
- Associate transactions with products.

FR-5 Smart Reorder Recommendations

The system shall identify products requiring replenishment using:

- Current stock.
- Minimum stock level.
- Historical stock-out demand.
- Supplier lead time.
- Safety stock.

FR-6 Analytics and Reporting

The system shall provide:

- Total products.
- Total inventory quantity.
- Low-stock information.
- Out-of-stock information.
- Inventory value information.
- Transaction statistics.

The system shall generate CSV and text reports.

FR-7 Background Monitoring

The system shall provide a background inventory monitoring mechanism that periodically checks inventory conditions and can identify low-stock situations.

8. NON-FUNCTIONAL REQUIREMENTS

8.1 Usability

The application shall provide a simple menu-driven CLI interface that can be used without a graphical environment.

8.2 Reliability

Inventory updates shall be performed using database transactions so that failed operations can be rolled back.

8.3 Concurrency

Concurrent inventory operations shall be protected using synchronization and database locking mechanisms.

8.4 Persistence

Warehouse information shall persist between application executions using the local H2 database.

8.5 Maintainability

The system shall use separate packages for models, repositories, services, UI, database, exceptions, I/O, and threading.

8.6 Portability

The project uses Maven and the Maven Wrapper so it can be executed on supported systems with Java 25.

8.7 Error Handling

Invalid user input and invalid business operations shall be handled using validation and custom exceptions.

8.8 Resource Efficiency

The application is lightweight and does not require an external database server, web server, or cloud infrastructure.

9. SYSTEM FEATURES AND MODULES

The application provides the following major modules.

9.1 Product Management

This module handles product creation, searching, filtering, updating, and deletion.

Each product contains information such as:

- Product ID.
- Product name.
- SKU.
- Category.
- Description.
- Unit price.
- Quantity.
- Minimum stock level.
- Supplier.

9.2 Inventory Management

This module handles:

- Stock In.
- Stock Out.
- Returns.
- Stock Adjustments.

All inventory changes are associated with stock transactions.

9.3 Supplier Management

Supplier information includes:

- Supplier ID.
- Supplier name.
- Contact information.
- Lead time.

Supplier lead time is used by the reorder recommendation engine.

9.4 Stock Transactions

The transaction module maintains the movement history of warehouse products.

Supported transaction types include:

- STOCK_IN

- STOCK_OUT
- RETURN
- ADJUSTMENT

The system uses inheritance and polymorphism for transaction types.

9.5 Smart Reorder Engine

The reorder engine analyzes inventory and historical stock-out information to recommend replenishment.

9.6 Warehouse Analytics

The analytics module calculates useful warehouse-level statistics and inventory information.

9.7 Report Generation

Reports are generated in:

- CSV format.
- Text format.

9.8 Background Inventory Monitor

The background monitor uses Java multithreading to periodically inspect inventory conditions.

10. TECHNOLOGY STACK

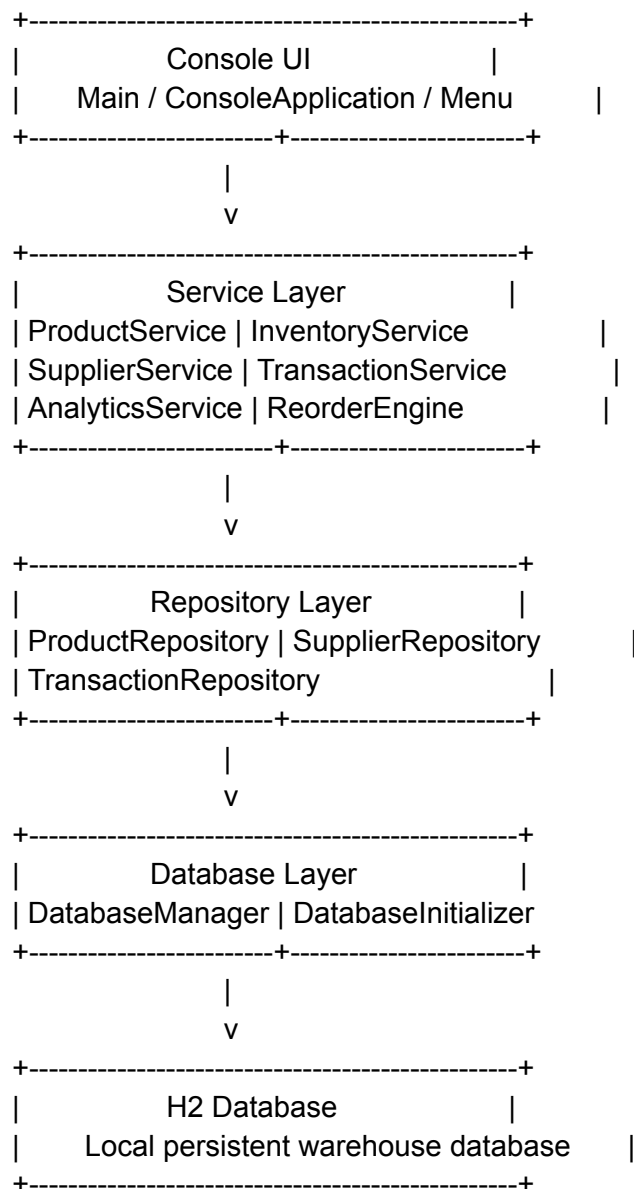
Technology	Purpose
Java 25	Main programming language
Maven	Build and dependency management
H2 Database	Embedded local persistence
JDBC	Database connectivity
JUnit 5	Unit testing
Java Collections	Data processing

Java Streams	Filtering and aggregation
Java I/O	Report generation
Multithreading	Background monitoring and concurrency
Git/GitHub	Version control

The project uses Java 25 and does not require an external database server.

11. SYSTEM ARCHITECTURE

The application follows a layered architecture.



Additional components include:

InventoryMonitor

|

v

InventoryService

ReorderRecommendationEngine

|

+---- ProductRepository

+---- SupplierRepository

+---- TransactionRepository

ReportGenerator

|

+---- CsvReportGenerator

+---- TextReportGenerator

This architecture separates user interaction, business logic, database access, and persistence.

12. SYSTEM WORKFLOW

The general workflow is:

Start Application

|

v

Initialize H2 Database

|

v

Create Schema / Load Existing Data

|

v

Display Main Menu

|

+--> Product Management

|

+--> Inventory Management

|

+--> Supplier Management

|

+--> Transaction History
|
+--> Reorder Recommendations
|
+--> Analytics
|
+--> Reports
|
+--> Background Monitor
|
v
Exit Application

For an inventory operation:

User selects Stock Operation

|
v
Select Product
|
v
Validate Input
|
v
Check Inventory Rules
|
v
Start Database Transaction
|
v
Update Product Quantity
|
v
Create Stock Transaction
|
v
Commit Transaction
|
v
Display Result

13.DIAGRAM

USE CASE DIAGRAM

Warehouse User interactions with the Smart Warehouse Stock Optimization System.

WAREHOUSE USER		SYSTEM BOUNDARY: SMART WAREHOUSE STOCK OPTIMIZATION SYSTEM	USE CASES
		→ Manage Products	Manage Products
		→ Manage Suppliers	Manage Suppliers
		→ Manage Inventory	Manage Inventory
		→ View Transactions	View Transactions
		→ View Reorder Recommendations	View Reorder Recommendations
		→ View Analytics	View Analytics
		→ Generate Reports	Generate Reports
		→ Enable Inventory Monitor	Enable Inventory Monitor

Actor: Warehouse User | Association: interacts with all eight use cases.

SEQUENCE DIAGRAM — STOCK-OUT OPERATION

Project flow: validate input → check product → update inventory → create transaction → commit → display result.

#	Warehouse User	Console UI	InputValidator	InventoryService	ProductRepository	TransactionRepository	H2 Database
1	→ Select Stock-Out						
2		→ validateInput()	→ validate()				
3				→ stockOut(productId, qty)			
4				→ findById(productId)	→ findById()		
5							→ SELECT product
6					← product details		← product details
7				→ UPDATE quantity			→ UPDATE PRODUCT
8				→ create StockOutTransaction		→ save()	
9						→ INSERT transaction	→ INSERT
10				← operation success		← transaction saved	
11		← display result	← validation/result				

Implementation note: The flow uses Console UI, InputValidator, InventoryService, repositories and JDBC/H2 persistence. Stock changes and transaction recording are handled within database transaction logic.

CLASS DIAGRAM

Major classes from the Java CLI implementation and their relationships.

Product	Supplier	StockTransaction <<abstract>>
- id: Long - name: String - sku: String - category: String - description: String	- id: Long - name: String - contact: String - leadTimeDays: Integer	- id: Long - productId: Long - transactionType: TransactionType - quantity: Integer - transactionDate: LocalDateTime

- unitPrice: Double - quantity: Integer - minStockLevel: Integer - supplierId: Long		- description: String + getTransactionDetails(): String
--	--	--

Relationships

1 Supplier	=====	* Product
1 Product	=====	* StockTransaction
StockTransaction	↳ inheritance ↳	StockInTransaction StockOutTransaction ReturnTransaction AdjustmentTransaction

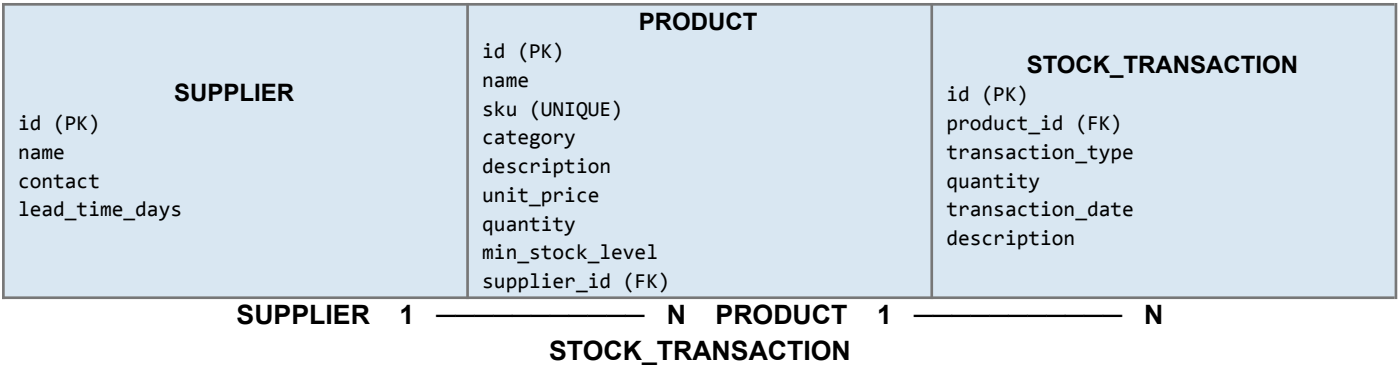
Service and persistence layer

ProductService + addProduct() + updateProduct() + deleteProduct() + searchProducts()	InventoryService + stockIn() + stockOut() + returnStock() + adjustStock()	SupplierService + addSupplier() + updateSupplier() + deleteSupplier() + searchSuppliers()	TransactionService + getTransactions() + getTransactionById() + getTransactionHistory()
ProductRepository + save() + findById() + findAll()	SupplierRepository + save() + findById() + findAll()	TransactionRepository + save() + findById() + findByProduct()	ReportGenerator <<interface>> + generate() CsvReportGenerator TextReportGenerator

Additional classes: AnalyticsService, ReorderRecommendationEngine, InventoryMonitor, DatabaseManager, DatabaseInitializer.

ENTITY-RELATIONSHIP (ER) DIAGRAM

Embedded H2 database design used by the Smart Warehouse Stock Optimization System.



Foreign keys: PRODUCT.supplier_id → SUPPLIER.id | STOCK_TRANSACTION.product_id → PRODUCT.id

Database: Embedded H2 file database at ./data/warehouse

14. DATABASE DESIGN

The application uses an embedded H2 database.

Database location:

./data/warehouse

The database is initialized automatically when the application starts.

14.1 Supplier Table

The SUPPLIER table stores supplier information.

Main fields include:

- ID
- NAME
- CONTACT
- LEAD_TIME_DAYS

14.2 Product Table

The PRODUCT table stores warehouse product information.

Main fields include:

- ID
- NAME
- SKU
- CATEGORY
- DESCRIPTION
- UNIT_PRICE
- QUANTITY
- MIN_STOCK_LEVEL
- SUPPLIER_ID

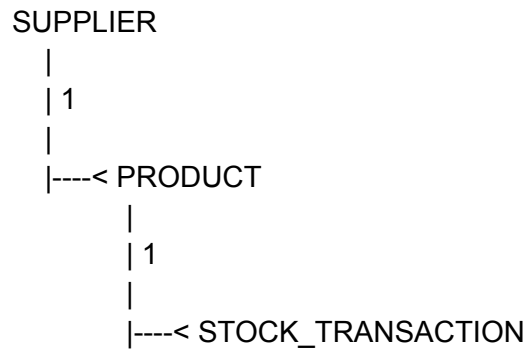
14.3 Stock Transaction Table

The STOCK_TRANSACTION table stores stock movement information.

Main fields include:

- ID
- PRODUCT_ID
- TRANSACTION_TYPE
- QUANTITY
- TRANSACTION_DATE
- DESCRIPTION

Relationship:



A supplier can supply multiple products.

A product can have multiple stock transactions.

15. OBJECT-ORIENTED DESIGN

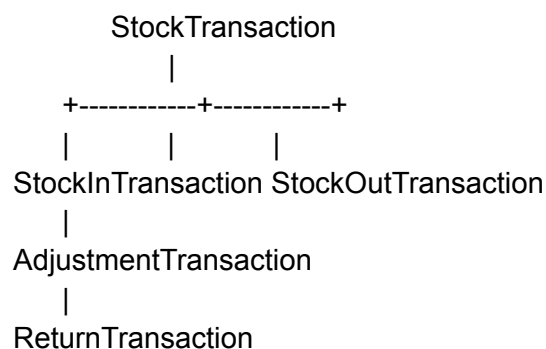
The project applies object-oriented programming throughout the implementation.

15.1 Encapsulation

Product, Supplier, and transaction classes encapsulate their fields using private attributes and controlled access through methods.

15.2 Inheritance

The transaction hierarchy contains:



The specific transaction classes inherit common behavior from the base `StockTransaction` class.

15.3 Polymorphism

Transactions can be handled through the common `StockTransaction` type while executing behavior associated with the specific transaction implementation.

The application also provides a detailed polymorphic transaction history option.

15.4 Interfaces

The report generation design uses the `ReportGenerator` interface.

Implementations include:

`ReportGenerator`

```
|
+---- CsvReportGenerator
|
+---- TextReportGenerator
```

This allows report generation to be extended without changing the core service logic.

15.5 Enumerations

Enums are used for controlled values such as:

- `TransactionType`
- `ReorderPriority`

This prevents invalid textual values from being used throughout the application.

15.6 Constructors and Method Overloading

Constructors are used to initialize domain objects, while overloaded methods support different forms of object creation or service operations where appropriate.

16. CSE2006 CONCEPT MAPPING

The project applies concepts from the Programming in Java syllabus.

CSE2006 Concept

Project Application

Java Fundamentals Main application, control flow, methods

Classes and Objects	Product, Supplier, Transaction classes
Encapsulation	Private fields and controlled access
Constructors	Domain object initialization
Inheritance	StockTransaction hierarchy
Method Overriding	Specialized transaction behavior
Polymorphism	Generic transaction processing
Abstract Classes	Transaction hierarchy
Interfaces	ReportGenerator
Enums	TransactionType and ReorderPriority
Exception Handling	Custom warehouse exceptions
Collections	Product/transaction data processing
Streams	Filtering and analytics
Java I/O	CSV and text report generation
JDBC	H2 database operations
Transactions	Atomic inventory updates
Multithreading	Background InventoryMonitor
Synchronization	Concurrent inventory operations

JPA is not implemented in this CLI version because the project uses JDBC directly for persistence.

17. SMART REORDER RECOMMENDATION ALGORITHM

The reorder recommendation engine determines whether a product requires replenishment.

The algorithm considers:

1. Current stock.
2. Minimum stock level as safety stock.

3. Historical stock-out quantity.
4. Historical demand period.
5. Supplier lead time.

17.1 Observed Daily Demand

The system calculates observed daily demand as:

$$\text{Daily Demand} = \frac{\text{Total Stock-Out Quantity}}{\text{Inclusive Calendar-Day Range}}$$

The calculated value is rounded upward.

If no stock-out history exists, observed demand is considered zero.

17.2 Reorder Point

The reorder point is:

$$\text{Reorder Point} = (\text{Daily Demand} \times \text{Supplier Lead Time}) + \text{Safety Stock}$$

If a product has no supplier, a seven-day fallback lead time is used.

17.3 Recommendation Condition

A reorder recommendation is generated when:

$$\text{Current Stock} \leq \text{Reorder Point}$$

17.4 Suggested Quantity

The suggested quantity is calculated using:

$$\text{Suggested Quantity} = \text{Reorder Point} + \text{Safety Stock} - \text{Current Stock}$$

17.5 Priority

Recommendations are categorized into:

- CRITICAL – current stock is zero.

- HIGH – current stock is at or below safety stock.
- MEDIUM – current stock is below reorder point.
- LOW – current stock is at reorder point but above safety stock.

17.6 Example

Assume:

Stock-out quantity = 20 units

Observed period = 5 days

Supplier lead time = 3 days

Safety stock = 8 units

Current stock = 15 units

Daily demand:

$$20 / 5 = 4 \text{ units/day}$$

Reorder point:

$$(4 \times 3) + 8 \\ = 20 \text{ units}$$

Suggested quantity:

$$20 + 8 - 15 \\ = 13 \text{ units}$$

Therefore, the product receives a reorder recommendation for approximately **13 units** with **MEDIUM** priority.

18. MULTITHREADING AND SYNCHRONIZATION

The system contains a background inventory monitor implemented using Java's `Runnable` mechanism.

The monitor runs independently from the main console workflow and checks inventory conditions.

The system also protects concurrent stock operations.

Inventory updates use:

- Per-product synchronization.
- Database transactions.
- Row-level locking using `SELECT ... FOR UPDATE`.

This prevents two concurrent stock operations from incorrectly modifying the same product quantity.

For example, if multiple stock-out requests compete for limited stock, the system validates the available quantity while the product record is locked.

This demonstrates the CSE2006 concepts of:

- Threads.
- Runnable.
- Synchronization.
- Concurrency.
- Database transactions.

19. EXCEPTION HANDLING AND INPUT VALIDATION

The project uses custom exceptions to represent business and database errors.

Examples include:

- `InvalidInputException`
- `ProductNotFoundException`
- `SupplierNotFoundException`
- `DuplicateSkuException`
- `InsufficientStockException`
- `InvalidStockOperationException`
- `DatabaseException`
- `WarehouseException`

Input validation is performed before executing business operations.

Examples of validation include:

- Invalid numeric values.
- Missing required values.

- Duplicate SKU.
- Non-existent product.
- Non-existent supplier.
- Invalid stock quantity.
- Stock-out greater than available quantity.

This prevents invalid data from reaching the database and improves system reliability.

20. FILE I/O AND REPORT GENERATION

The application provides report generation using Java I/O.

Reports are generated in:

`./reports/`

Generated reports include:

`inventory_report.csv`
`low_stock_report.csv`
`transaction_report.txt`
`warehouse_summary.txt`

CSV reports can be opened using spreadsheet applications, while text reports provide human-readable summaries.

The report-generation architecture uses an interface:

`ReportGenerator`

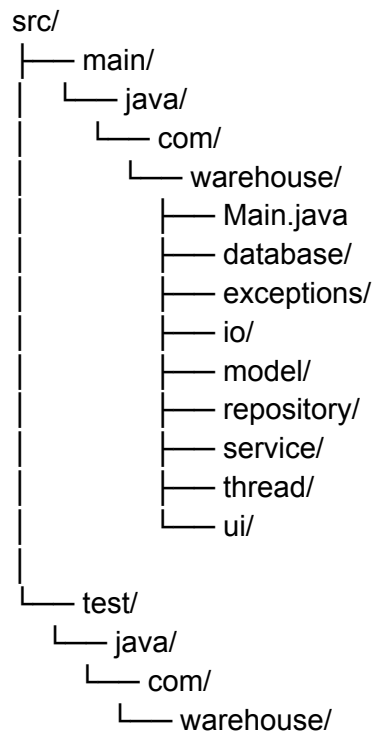
with implementations:

`CsvReportGenerator`
`TextReportGenerator`

This demonstrates interface-based design and Java file I/O.

21. IMPLEMENTATION DETAILS

The project is organized into packages according to responsibility.



Important classes include:

Main

Application entry point.

ConsoleApplication

Controls the main application lifecycle.

ConsoleMenu

Displays and processes menu options.

ProductService

Handles product-related business logic.

InventoryService

Handles stock operations and synchronized inventory updates.

SupplierService

Handles supplier operations.

TransactionService

Handles transaction processing.

ReorderRecommendationEngine

Calculates stock replenishment recommendations.

AnalyticsService

Provides warehouse statistics.

InventoryMonitor

Provides background inventory monitoring.

ProductRepository

Provides JDBC-based product persistence.

SupplierRepository

Provides JDBC-based supplier persistence.

TransactionRepository

Provides JDBC-based transaction persistence.

DatabaseInitializer

Creates the database schema and initial sample data.

22. TESTING AND VALIDATION

The project uses **JUnit 5** for automated testing.

The final test suite contains:

23 tests
0 failures
0 errors

Tests cover important parts of the application.

22.1 Product Service Testing

Tests include:

- Product creation.
- Product retrieval.
- Product search.
- Product validation.
- Duplicate SKU handling.

22.2 Supplier Service Testing

Tests include:

- Supplier creation.
- Supplier retrieval.
- Supplier validation.

22.3 Inventory Testing

Tests include:

- Stock-in.
- Stock-out.
- Return.
- Adjustment.
- Insufficient stock validation.
- Transaction rollback.

22.4 Reorder Engine Testing

Tests cover:

- Low-stock products.
- Out-of-stock products.
- Reorder-point boundaries.
- Suggested reorder quantities.
- Supplier lead time.

22.5 Concurrency Testing

The concurrency test creates multiple stock-out tasks competing for limited inventory.

The test verifies that synchronized inventory operations prevent the available quantity from becoming invalid.

22.6 Analytics Testing

Tests cover normal inventory data and empty-data conditions.

22.7 Report Testing

Report generation tests verify CSV and text report creation, including CSV escaping behavior.

23. Running the Project and RESULTS AND OUTPUT

Running the Project

The Smart Warehouse Stock Optimization System can be executed directly through Visual Studio Code using the Java Extension Pack. The project does not require the evaluator to run `mvnw.cmd` `exec:java` to launch the application.

Option 1 — Run Directly from Visual Studio Code (Recommended for Evaluation)

Follow the steps below to run the application:

1. Open the project folder in Visual Studio Code.
2. Ensure that **JDK 25** is installed and properly configured.
3. Navigate to the main application file, `Main.java`.
4. Locate the `main()` method in the file.
5. Click the **Run ▶** button displayed above the `main()` method.
6. Select **Run Java** to compile and launch the application.

The Java Extension Pack automatically compiles the required project classes and starts the console-based application.

Application Main Menu

Once the application starts, the following main menu is displayed:

SMART WAREHOUSE STOCK OPTIMIZATION SYSTEM

1. Product Management
2. Inventory Management
3. Supplier Management
4. Stock Transactions & History
5. Smart Reorder Recommendations
6. Warehouse Analytics
7. Generate Reports
8. START Background Stock Monitor
9. View Detailed Transaction History
0. Exit Application

Evaluator Note: The recommended evaluation method is to open `Main.java` in Visual Studio Code and use the Run button. Maven Wrapper commands are also provided for automated testing and project builds.

Option 2 — Run Using Maven Wrapper

The complete Maven project can also be executed through the terminal using the Maven Wrapper.

Windows

```
mvnw.cmd clean test
```

```
mvnw.cmd exec:java
```

Linux / macOS

```
./mvnw clean test
```

```
./mvnw exec:java
```

The `clean test` command cleans the project, compiles the source code, and executes the automated tests. The `exec:java` command launches the application through Maven.

Running Tests

The project includes **23 JUnit 5 automated tests** covering the following functional and technical areas:

- Product management
- Inventory operations
- Supplier management
- Smart reorder recommendations
- Warehouse analytics
- Report generation
- Concurrency
- Exception handling
- Transaction safety

These tests help verify the correctness, reliability, and stability of the system.

Executing Tests

Windows

Open the project folder in the terminal and execute:

```
mvnw.cmd clean test
```

Linux / macOS

```
./mvnw clean test
```

Expected Test Result

A successful test execution should display a result similar to the following:

Tests run: 23, Failures: 0, Errors: 0

BUILD SUCCESS

Note: The expected result indicates that all 23 tests have passed without failures or errors. The actual console output may vary slightly depending on the Maven and Surefire plugin versions.

The application successfully initializes the local database and loads sample warehouse data.

```
=====
SMART WAREHOUSE STOCK OPTIMIZATION SYSTEM
=====
1. Product Management
2. Inventory Management (Stock In/Out/Return/Adjust)
3. Supplier Management
4. Stock Transactions & History
5. Smart Reorder Recommendations
6. Warehouse Analytics
7. Generate Reports (CSV & Text)
8. START [Stopped] Background Stock Monitor
9. View Detailed Transaction History (Polymorphic)
0. Exit Application
=====

Enter your choice: 1

--- PRODUCT MANAGEMENT ---
1. Add New Product
2. View All Products
3. Search Product by Name
4. Search Product by SKU
5. Filter Products by Category
6. Update Product Details
7. Delete Product
0. Back to Main Menu

Enter choice: 2
```

```
-----
ID | NAME | SKU | CATEGORY | PRICE ($) | QTY | MIN_STK | STATUS
-----
1 | Wireless Mouse | SKU-1001 | Electronics | 25.50 | 45 | 15 | OK
2 | Mechanical Keyboard | SKU-1002 | Electronics | 89.99 | 8 | 10 | LOW STOCK
3 | Barcode Scanner | SKU-1003 | Electronics | 120.00 | 0 | 5 | OUT OF STOCK
4 | USB Hub 4-Port | SKU-1004 | Accessories | 19.99 | 60 | 20 | OK
5 | Laptop Stand | SKU-1005 | Accessories | 34.50 | 12 | 15 | LOW STOCK
6 | Ethernet Cable 10m | SKU-1006 | Networking | 12.00 | 100 | 25 | OK
7 | Thermal Printer | SKU-1007 | Electronics | 210.00 | 3 | 5 | LOW STOCK
8 | Storage Drive 2TB | SKU-1008 | Storage | 75.00 | 22 | 10 | OK
9 | Safety Gloves | SKU-1009 | Warehouse Gear | 8.50 | 0 | 30 | OUT OF STOCK
10 | Packaging Box Large | SKU-1010 | Packaging | 2.50 | 500 | 100 | OK
11 | Packing Tape Roll | SKU-1011 | Packaging | 4.00 | 15 | 50 | LOW STOCK
12 | Barcode Label Roll | SKU-1012 | Packaging | 15.00 | 80 | 20 | OK
-----

--- PRODUCT MANAGEMENT ---
1. Add New Product
2. View All Products
3. Search Product by Name
4. Search Product by SKU
5. Filter Products by Category
6. Update Product Details
7. Delete Product
0. Back to Main Menu

Enter choice: 1

--- ADD NEW PRODUCT ---
Product Name: Wireless Headset
Product SKU: SKU-1013
Category: Electronics
Description: Wireless headset for warehouse staff communication
Unit Price ($): 59.99
Initial Quantity: 10
Minimum Stock Level: 15
```

```
Available Suppliers:
[ID: 1] TechLogistics Inc (Lead: 5 days)
[ID: 2] Global Components Co (Lead: 7 days)
[ID: 3] ConnectPro Supplies (Lead: 3 days)
[ID: 4] Industrial Safety Gear (Lead: 10 days)
[ID: 5] Packaging World (Lead: 4 days)
```

```
Supplier ID (Press Enter to skip): 1
SUCCESS: Product added with ID #13
```

```
--- PRODUCT MANAGEMENT ---
```

1. Add New Product
2. View All Products
3. Search Product by Name
4. Search Product by SKU
5. Filter Products by Category
6. Update Product Details
7. Delete Product
0. Back to Main Menu

```
Enter choice: 0
```

```
=====
SMART WAREHOUSE STOCK OPTIMIZATION SYSTEM
=====
```

1. Product Management
2. Inventory Management (Stock In/Out/Return/Adjust)
3. Supplier Management
4. Stock Transactions & History
5. Smart Reorder Recommendations
6. Warehouse Analytics
7. Generate Reports (CSV & Text)
8. START [Stopped] Background Stock Monitor
9. View Detailed Transaction History (Polymorphic)
0. Exit Application

```
Enter your choice: 0
```

```
Exiting Smart Warehouse Stock Optimization System. Goodbye!
```

```
=====
SMART WAREHOUSE STOCK OPTIMIZATION SYSTEM
=====
```

1. Product Management
2. Inventory Management (Stock In/Out/Return/Adjust)
3. Supplier Management
4. Stock Transactions & History
5. Smart Reorder Recommendations
6. Warehouse Analytics
7. Generate Reports (CSV & Text)
8. START [Stopped] Background Stock Monitor
9. View Detailed Transaction History (Polymorphic)
0. Exit Application

```
Enter your choice: 3
```

```
--- SUPPLIER MANAGEMENT ---
```

1. Add New Supplier
2. View All Suppliers
3. Search Supplier by Name
4. Update Supplier
5. Delete Supplier
0. Back to Main Menu

```
Enter choice: 2
```

```
Supplier[ID=1, Name='TechLogistics Inc', Contact='John Doe', Email='john@techlogistics.com', LeadTime=5 days, Score=4.8/5.0]
Supplier[ID=2, Name='Global Components Co', Contact='Jane Smith', Email='jane@globalcomp.com', LeadTime=7 days, Score=4.5/5.0]
Supplier[ID=3, Name='ConnectPro Supplies', Contact='Bob Cable', Email='sales@connectpro.com', LeadTime=3 days, Score=4.2/5.0]
Supplier[ID=4, Name='Industrial Safety Gear', Contact='Alice Guard', Email='support@safetymfg.com', LeadTime=10 days, Score=3.9/5.0]
Supplier[ID=5, Name='Packaging World', Contact='Pack Team', Email='orders@packworld.com', LeadTime=4 days, Score=4.7/5.0]
```

=====

SMART WAREHOUSE STOCK OPTIMIZATION SYSTEM

=====

1. Product Management
 2. Inventory Management (Stock In/Out/Return/Adjust)
 3. Supplier Management
 4. Stock Transactions & History
 5. Smart Reorder Recommendations
 6. Warehouse Analytics
 7. Generate Reports (CSV & Text)
 8. START [Stopped] Background Stock Monitor
 9. View Detailed Transaction History (Polymorphic)
 0. Exit Application
- =====

Enter your choice: 5

=====

SMART REORDER RECOMMENDATIONS

=====

REORDER RECOMMENDATION

Product: Barcode Scanner (SKU: SKU-1003) [ID: 3]
Current Stock: 0
Average Daily Demand: 10 units/day
Supplier Lead Time: 5 days
Safety Stock: 5
Reorder Point: 55
Recommended Quantity: 60
Priority: CRITICAL

Reason: 10 units were dispatched in 1 recorded stock-out transaction(s), averaging 10 units/day. The reorder point covers 5 supplier lead-time day(s) plus safety stock.

REORDER RECOMMENDATION

Product: Safety Gloves (SKU: SKU-1009) [ID: 9]
Current Stock: 0
Average Daily Demand: 0 units/day
Supplier Lead Time: 10 days
Safety Stock: 30
Reorder Point: 30

Reorder Point: 30
Recommended Quantity: 60
Priority: CRITICAL

Reason: Current stock is at or below the configured safety-stock threshold of 30; no stock-out history is available.

REORDER RECOMMENDATION

Product: Mechanical Keyboard (SKU: SKU-1002) [ID: 2]
Current Stock: 8
Average Daily Demand: 12 units/day
Supplier Lead Time: 5 days
Safety Stock: 10
Reorder Point: 70
Recommended Quantity: 72
Priority: HIGH

Reason: 12 units were dispatched in 1 recorded stock-out transaction(s), averaging 12 units/day. The reorder point covers 5 supplier lead-time day(s) plus safety stock.

REORDER RECOMMENDATION

Product: Laptop Stand (SKU: SKU-1005) [ID: 5]
Current Stock: 12
Average Daily Demand: 0 units/day
Supplier Lead Time: 7 days
Safety Stock: 15
Reorder Point: 15
Recommended Quantity: 18
Priority: HIGH

Reason: Current stock is at or below the configured safety-stock threshold of 15; no stock-out history is available.

REORDER RECOMMENDATION

Product: Thermal Printer (SKU: SKU-1007) [ID: 7]
Current Stock: 3
Average Daily Demand: 0 units/day
Supplier Lead Time: 5 days
Safety Stock: 5
Reorder Point: 5
Recommended Quantity: 7
Priority: HIGH

```
Reason: Current stock is at or below the configured safety-stock threshold of 5; no stock-out history is available.
-----
REORDER RECOMMENDATION
-----
Product: Packing Tape Roll (SKU: SKU-1011) [ID: 11]
Current Stock: 15
Average Daily Demand: 0 units/day
Supplier Lead Time: 4 days
Safety Stock: 50
Reorder Point: 50
Recommended Quantity: 85
Priority: HIGH
Reason: Current stock is at or below the configured safety-stock threshold of 50; no stock-out history is available.
-----
REORDER RECOMMENDATION
-----
Product: Wireless Headset (SKU: SKU-1013) [ID: 13]
Current Stock: 10
Average Daily Demand: 0 units/day
Supplier Lead Time: 5 days
Safety Stock: 15
Reorder Point: 15
Recommended Quantity: 20
Priority: HIGH
Reason: Current stock is at or below the configured safety-stock threshold of 15; no stock-out history is available.
-----
=====
```

24. DESIGN DECISIONS

24.1 Technology Selection

Java 25 was selected as the primary development language because the project is designed to demonstrate concepts covered in CSE2006, including object-oriented programming, collections, exceptions, JDBC, file I/O, and multithreading.

H2 was selected as the database because it provides persistent local database storage without requiring an external database server.

JDBC was used for database access to directly demonstrate SQL queries, prepared statements, transactions, and database connectivity concepts covered in the course.

24.2 Application Design

A **CLI-based interface** was chosen to keep the application lightweight, portable, and focused on Java programming and backend functionality rather than frontend development.

A **layered architecture** was adopted to separate the user interface, business logic, repository/database access, and persistence components. This improves maintainability, testability, and separation of responsibilities.

25. CHALLENGES FACED

Several challenges were considered during development.

Database Consistency

Inventory updates need to modify both the product quantity and transaction history correctly.

This was addressed using JDBC transactions and rollback handling.

Concurrent Inventory Updates

Multiple inventory operations may attempt to modify the same product.

This was addressed using synchronization and database row locking.

Reorder Calculation

Reorder recommendations require combining current inventory, safety stock, historical demand, and supplier lead time.

A deterministic reorder algorithm was implemented.

Input Validation

A CLI application receives user input directly from the console, which can contain invalid values.

Validation and custom exceptions were added to handle these conditions.

Maintainability

The project contains multiple functional areas.

A layered package structure was used to keep responsibilities separated.

26. LIMITATIONS

The current system has several limitations:

1. It is designed for a single local warehouse environment.
2. It does not provide user authentication.
3. It does not provide role-based permissions.
4. It does not integrate with external supplier systems.

5. Demand forecasting is based on recorded stock-out history rather than machine learning.
 6. The demand-analysis window is not configurable.
 7. Database schema migration is not implemented.
 8. The application does not provide a graphical interface in this CLI version.
 9. The application does not expose a REST API.
 10. JPA is not implemented because the project uses JDBC directly.
-

27. FUTURE ENHANCEMENTS

Possible future enhancements include:

- Multi-user authentication.
 - Role-based access control.
 - Multiple warehouse locations.
 - Web-based dashboard.
 - REST API.
 - Mobile application.
 - Supplier integration.
 - Email notifications.
 - Configurable demand-analysis periods.
 - Advanced demand forecasting.
 - Machine-learning-based inventory prediction.
 - Automatic purchase-order generation.
 - Database migration support.
 - Advanced visualization and charts.
 - Cloud deployment.
-

28. LEARNING OUTCOMES

The project provided practical experience in applying Java programming concepts to a real-world problem.

The major learning outcomes include:

Object-Oriented Programming

The project demonstrated classes, objects, encapsulation, inheritance, abstraction, interfaces, overriding, and polymorphism.

Exception Handling

Custom exceptions were used to handle invalid business operations and database errors.

Collections and Streams

Collections and streams were used for data processing, filtering, and analytics.

Multithreading

A background inventory monitor and concurrent inventory operations demonstrated thread-related concepts.

Synchronization

Synchronization and database locking were used to maintain inventory consistency.

Java I/O

Java I/O was used for generating CSV and text reports.

JDBC

JDBC was used to communicate directly with the H2 database.

Database Transactions

Database transactions were used to ensure atomic inventory updates.

Software Design

Layered architecture and separation of responsibilities improved the maintainability of the application.

29. CONCLUSION

The **Smart Warehouse Stock Optimization System – Java CLI Version** successfully provides a lightweight and self-contained solution for warehouse inventory management.

The system supports product and supplier management, inventory transactions, transaction history, reorder recommendations, analytics, report generation, and background inventory monitoring.

A major strength of the system is that it goes beyond basic CRUD operations. The reorder recommendation engine combines inventory levels, safety stock, historical demand, and supplier lead time to identify products that require replenishment.

The project also demonstrates a wide range of CSE2006 Programming in Java concepts, including object-oriented programming, inheritance, polymorphism, interfaces, exception handling, collections, streams, I/O, JDBC, multithreading, synchronization, and database transactions.

The use of Java 25, Maven, JUnit 5, JDBC, and an embedded H2 database makes the project easy to run locally while keeping the implementation technically substantial.

The project therefore provides a practical demonstration of Java programming concepts through a real-world warehouse management problem.

20. REFERENCES

1. CSE2006 – Programming in Java course syllabus and laboratory topics.
 2. VITyarthi project instructions and evaluation guidelines.
 3. Java Platform documentation and Java language concepts.
 4. JDBC documentation and database connectivity concepts.
 5. H2 Database documentation.
 6. JUnit 5 testing framework documentation.
 7. Maven documentation.
-